Centro Universitário de Excelência
Sistemas de Informação

Docker, Docker-Compose e Kubernetes

Autor: Erick dos Santos Rezende
Kailla de Souza Costa
Laysa Lima de Pinho
Samira de Jesus Santos
Vinícius de Oliveira Costa
Victor Rogerio Aguiar do Rosário

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é percorrer as decisões técnicas do isolamento em containers até a orquestração do Kubernetes.

1. Contexto do problema:
Instabilidade do sistema legado da MecâniQA Tech

2. Docker:
Dockerfile e isolamento da aplicação Java

3. Docker-Compose:
Orquestração local, rede e persistência

4. Kubernetes:
Deployments, Services e auto-healing em produção

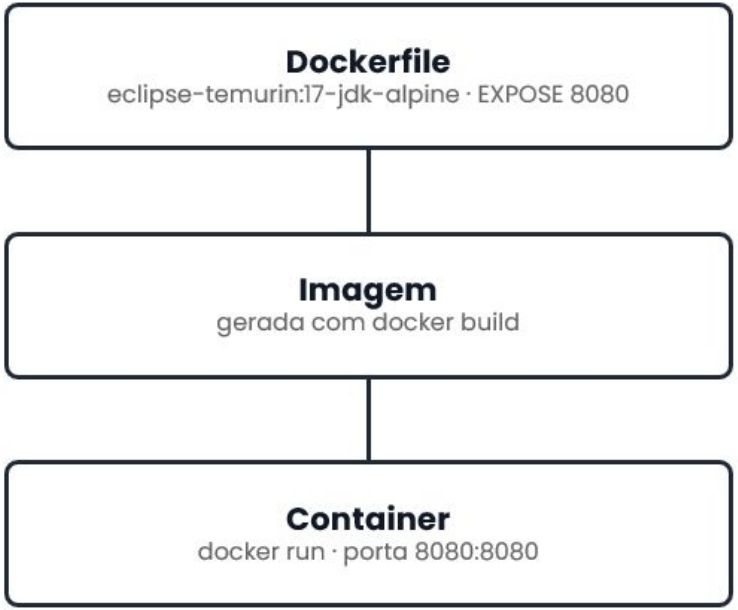
5. Considerações finais:
Resultados obtidos

Contexto do problema

- Servidor físico legado instável;
- A API Java consome todos os recursos e derruba ao mesmo tempo o MySQL e o Redis;
- A infraestrutura sofre para escalar e os recursos são frequentemente esgotados.;

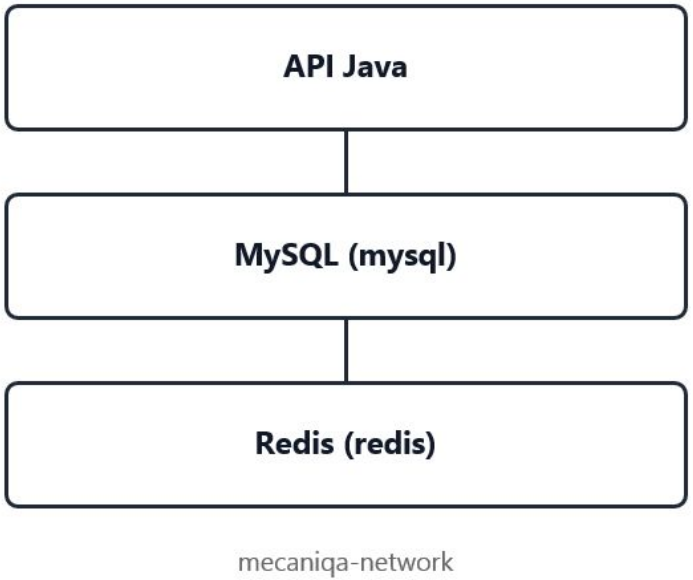
Docker

- Imagem base: eclipse-temurin:17--jdk-alpine;
- Mais leve que uma VM: containers compartilham o kernel do servidor, sem um SO completo por aplicação;
- EXPOSE 8080 indica a porta, o mapeamento 8080:8080 (-p) a disponibiliza;
- ENTRYPOINT mantém a API em primeiro plano;



Docker-Compose

- Comunicação interna (DNS): MySQL e Redis resolvidos por nome do serviço, sem depender de IPs fixos;
- Persistência de dados (Volumes): Os dados sobrevivem a reinícios;
- Todo o ambiente sobe com um único comando

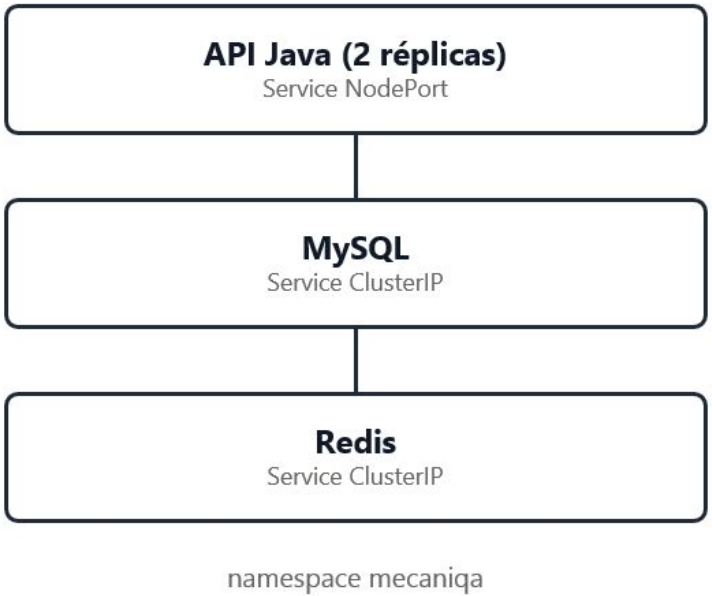


Docker-Compose: evidência

```
user@DESKTOP-GPHD5DH:/mnt/c/Users/User/lab-banco$ curl.exe http://localhost:8080/api/status
curl.exe http://localhost:8080/actuator/health
{"aplicacao":"UP","timestamp":"2026-09-03T00:16:39.2733595567","mysql":"UP"}{"components":{"db":{"details":{"database":"
MySQL","validationQuery":"isValid()"},"status":"UP"},"diskSpace":{"details":{"total":1081101176832,"free":1019565244416,
"threshold":10485760,"path":"/app/.","exists":true},"status":"UP"},"livenessState":{"status":"UP"},"ping":{"status":"UP"
},"readinessState":{"status":"UP"},"redis":{"details":{"version":"7.4.11"},"status":"UP"},"ssl":{"details":{"expiringCha
ins":[],"invalidChains":[],"validChains":[]},"status":"UP"}},"groups":["liveness","readiness"],"status":"UP"}user@DESKTO
P-GPHD5DH:/mnt/c/Users/User/lab-banco$
```

Kubernetes

- Pod isolado é frágil, o Deployment mantém o estado desejado e recria o Pod que falhar (auto-healing)
- MySQL e Redis em ClusterIP: rede privada, só a API acessa
- API em NodePort: Atende quem vem de fora



Kubernetes: evidência

```
user@DESKTOP-GPHD5DH:/mnt/c/Users/User/lab-banco$ kubectl get pods,services,pvc -n mecaniqa
curl http://localhost:8081/api/status
curl http://localhost:8081/actuator/health
NAME                         READY   STATUS    RESTARTS   AGE
pod/api-56dc4d48d4-69hqf     1/1     Running   0          4m44s
pod/api-56dc4d48d4-k6td6     1/1     Running   0          4m44s
pod/mysql-7585bdf48c-ls4cp   1/1     Running   0          4m44s
pod/redis-74f9bf9cd6-q2ng9   1/1     Running   0          4m44s

NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/api     NodePort    10.96.251.253   <none>        8080:30080/TCP   4m44s
service/mysql   ClusterIP   10.96.227.118   <none>        3306/TCP         4m44s
service/redis   ClusterIP   10.96.208.85    <none>        6379/TCP         4m44s

NAME                                  STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEA
TTRIBUTESCLASS   AGE
persistentvolumeclaim/mysql-data      Bound    pvc-ef1d169a-0bdd-4877-a6ca-09e050313380   2Gi        RWO            standard       <unset>
                 4m44s
persistentvolumeclaim/redis-data      Bound    pvc-6bf359f6-bddf-4f83-a30e-59da7a52f156   1Gi        RWO            standard       <unset>
                 4m44s
{"aplicacao":"UP","timestamp":"2026-09-10T01:24:09.111810964Z","mysql":"UP"}{"components":{"db":{"details":{"database":"MySQL","validat
ionQuery":"isValid()"},"status":"UP"},"diskSpace":{"details":{"total":1081101176832,"free":1014866731008,"threshold":10485760,"path":"/
app/.","exists":true},"status":"UP"},"livenessState":{"status":"UP"},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"redis":{"
details":{"version":"7.4.11"},"status":"UP"},"ssl":{"details":{"expiringChains":[],"invalidChains":[],"validChains":[]},"status":"UP"}}
,"groups":["liveness","readiness"],"status":"UP"}user@DESKTOP-GPHuser@DESKTuser@DESKTuser@DESKTuser@user@user@user@user@user@user@user@
```

kubectl get pods,services,pvc -n mecaniqa

Considerações finais

- Dockerfile isolou a API Java, resolvendo a disputa de recursos no servidor legado;
- Docker Compose orquestrou três serviços com DNS interno e volumes persistentes;
- Kubernetes trouxe resiliência: auto-healing, duas réplicas da API e exposição controlada;
- Aplicação, MySQL e Redis validados com status UP em todas as camadas

Referências

- **AMAZON WEB SERVICES**. Qual a diferença entre imagens Docker e contêineres? AWS, [s. d.]. Disponível em: https://aws.amazon.com/pt/compare/the-difference-between-docker-images-and-containers/. Acesso em: 16 set. 2026.
- **DOCKER**. Guides. Docker Documentation, [s. d.]. Disponível em: https://docs.docker.com/guides/. Acesso em: 16 set. 2026.
- **HABBEMA**. Docker & Dockerfile. Medium, 20 mar. 2020. Disponível em: https://medium.com/@habbema/docker-dockerfile-28d89840570b. Acesso em: 16 set. 2026.
- **IXIMIUZ LABS**. Containers vs. Pods. iXimiuz Labs Tutorials, [s. d.]. Disponível em: https://labs.iximiuz.com/tutorials/containers-vs-pods. Acesso em: 16 set. 2026.
- **WARP**. Docker Compose Port Mapping. Warp Terminus, [s. d.]. Disponível em: https://www.warp.dev/terminus/docker-compose-port-mapping. Acesso em: 16 set. 2026.